EN.530.646
Robot Devices, Kinematics, Dynamic and Control
Lab 1*

Jin Seob Kim, Ph.D.
Associate Teaching Profesor, ME Dept., LCSR, JHU

Out: 10/20/2026, Tuesday
Due: 10/29 ~~27~~/2026, Thursday ~~Tuesday~~ by midnight ET

- *This is exclusively used for Fall 2026 EN.530.646 RDKDC students and is not to be posted, shared, or otherwise distributed.*
- *Using LLM or any other GenAI tools is not permitted, unless stated otherwise.*

There are two tasks in this Lab assignment.

TASK 1: Get Familiar with UR5(e) Simulation

During this task, you will be implementing simple control over the UR5(e) robot through R-VIZ.

Interface

Write a script titled `lab1_1` that controls the UR5(e) to move through a predefined sequence of at least **four** distinct joint configurations. You will use the following API functions to control the robot. (Note: The APIs for the UR5 and UR5e are identical; we are using the UR5e as an example in this task.)

Before proceeding, ensure that you have installed the `ROS Toolbox` and the `Robotics System Toolbox`, as described in the installation instructions in Lab 0.

1. `ur5e=ur_interface()` initializes an object to interface with UR5e via ROS2 Jazzy. ur5e has member functions to get data and send commands. It is recommended to use `pause()` with an appropriate delay to ensure proper connection.
2. `ur5e.move_joints(theta, time_interval)` sends the desired robot joint configuration to the robot, the robot will move accordingly after the command is sent.

   `theta`: the joint angles in [rad] as a $6 \times N$ matrix (N goals)

   `time_interval`: time between arriving goals in [sec].

*This document was originally developed by Prof. Noah Cowan.

For example, `ur5e.move_joints(ur5e.home,10)` will command UR5e to move to the home position in 10 seconds.

UR5e will move to the target by interpolating the joint angles. So the trajectory of each moving command is approximately a line in the configuration space, not a line in the Cartesian space.

NOTE that if calls to this function are made in rapid succession, the robot will probably not achieve intermediate waypoints – that is, the robot does not queue the locations, it replaces the current target with the new target. Feel free to play around with this e.g. by moving the robot back and forth between two locations with different lag between the calls. To fix this behaviour it is best practice to place a pause after each command equal to the time_interval of the command.

NOTE Make sure to handle exceptions properly in your MATLAB script. An unexpected crash may leave behind residual ROS2 connections, which could interfere with subsequent attempts to control the UR robot.

3. `theta=ur5e.get_current_joints()` reads the current robot joint angles.

Run

1. Complete your `lab1_1.m` code.
2. Open a terminal, type `ros2 launch rdkdc ur5e_simulation.launch.py` to start the simulation.
3. Open a new terminal, type `matlab` to launch MATLAB. Navigate to the folder where your code is and add `ur5e_interface` folder to the path.
4. Run `lab1_1.m`. You should see all commands executed sequentially by your simulated robot.

TASK 2: Synthesizing Rotation and Homogeneous Matrices

Review your lecture notes and the textbook (MLS) for the rotations about the X, Y, and Z axes.

1. Write a function `ROTX( )` which accepts a scalar roll value (in radians) and returns the corresponding 3×3 rotation matrix.

   The following help functions are available under `Matlabcodes_Lab1/` on Canvas. Download them to your code directory. For each task (a)-(f) print and annotate enough plots to convince yourself (and the TAs) that your functions work, i.e. show that rotations and translations in each direction and combinations of them are working as expected. This can take as little as one figure with 4 data points per question if you choose your transformations well.

   (a) The function `PLOTR( )` accepts a single 3×3 matrix and plots the columns of the matrix as vectors. Print and annotate plots that show various rotations about the ROLL axis.

   (b) The function `PLOTP3( )` accepts a single 3×1 vector and plots the 3×1 vector as a point. Print and annotate plots that show various points and transformed points.

   (c) The function `PLOTP4( )` accepts a single 4×1 homogeneous vector and plots the 4×1 vector as a point. Print and annotate plots that show various points and transformed points.

   (d) The function `PLOTV3( )` accepts two 3×1 vectors and plots the vector connecting two 3×1 points. Print and annotate plots that show various vectors and transformed vectors.

   (e) The function `PLOTV4( )` accepts two 4×1 homogeneous vectors and plots the vector connecting two 4×1 points. Print and annotate plots that show various vectors and transformed vectors.

(f) The function `PLOTF( )` plots frames. Print and annotate plots that show various frames and transformed frames.

2. Check with `ROTY( )` and `ROTZ( )`. You don't need to submit plots for these two cases though. However, the MATLAB code should be included in the submission.

3. Use the above functions to write a function `EULERXYZ( )` which accepts one input, a 3-vector of angles (in radians), and returns the corresponding 3×3 rotation matrix. Experiment with plotting various general rotations.

4. Write a function `EULERXYZINV( )` which accepts a 3×3 rotation matrix and returns a 3×1 vector (in radians) using the formulas you derived in homework 3 (and also appears for various different Euler angle conventions in MLS textbook).

   Experiment with the function you wrote above, `EULERXYZ( )`, and the new function `EULERXYZINV( )`. Experiment with them to convince yourself (and the grader!) of the following:[1]

   (a) EULERXYZ([a b c]) equals EULERXYZ([a+(2π) b c]) for real numbers a, b, c.

   (b) For some rotation matrices R, the function EULERXYZINV(R) is numerically ill-defined. What is wrong? Include this answer in the comment section at the top of your EULERXYZINV(R) function. Add a "warning" that will display on the MATLAB terminal if such a condition is found, and add logic to handle such a condition gracefully. Note that MATLAB might automatically handle this numerical issue by itself. But *you have to specify this condition in your submission regardless of that.*

   (c) That [a b c] $\neq$ EULERXYZINV(EULERXYZ([a b c])) for some a, b, c — e.g. try [4π 0 0].

   (d) R = EULERXYZ(EULERXYZINV(R)) for all a,b,c where EULERXYZINV( ) is well-defined.

5. Write a function `SKEW3( )` which accepts a 3×1 vector $\mathbf{x} = [x_1, x_2, x_3]^\top$ and returns the corresponding canonical 3×3 skew-symmetric matrix given in class and in MLS.

6. Write a function `EXPCR( )` which accepts a 3×1 vector $\mathbf{x} = [x_1, x_2, x_3]^\top$. The function should return the 3×3 rotation matrix which represents a rigid-body rotation of angle $\|\mathbf{x}\|$ radians about the fixed axis $\mathbf{n} = \mathbf{x}/\|\mathbf{x}\|$. Your function should use `SKEW3( )` and Rodrigues' formula (FYI, using the built-in function `EXPM( )` will be the same). Note that when using Rodrigues' formula, your code has to be general enough to include all possible $\mathbf{x}$'s.

7. Use `EXPCR( )` to generate rotation matrices for rotations of various angles about several different fixed axes. For each example perform the following:

   (a) Use `PLOTR( )` to plot out the rotated and original frames.

   (b) Use `PLOTV3( )` to plot various vectors before and after being rotated.

8. Write a function `SKEW6( )` which accepts a 6×1 vector $\xi = [\mathbf{w}\,,\, \mathbf{v}]^\top$, where $\mathbf{w} = [\omega_1, \omega_2, \omega_3]$ and $\mathbf{v} = [v_1, v_2, v_3]$, returns the corresponding canonical 4×4 skew-symmetric matrix in se(3), also called matrix representation of twist, given in class and in MLS.

9. Write a function `EXPCF( )` which accepts a 6×1 vector $\xi = [\mathbf{w}\,,\, \mathbf{v}]^\top$, where $\mathbf{w} = [\omega_1, \omega_2, \omega_3]$ and $\mathbf{v} = [v_1, v_2, v_3]$ The function should return the 4×4 matrix which represents a rigid-body transformation. Your function should use `SKEW6( )`. *Hint:* You may extract θ the same way as in `EXPCR()`. Be sure to handle corner cases.

10. Use `EXPCF( )` to generate rotation matrices for transformations of various angles about several different fixed axes. For each example perform the following:

    (a) Use `PLOTF( )` to plot out the transformed and original frames.

    (b) Use `PLOTV3( )` to plot various vectors before and after being transformed.

[1] Note: we are looking for numerical evidence, not a theoretical proof!

11. Write and test `FINV( )` which accepts a 4×4 homogeneous transformation and returns its MATRIX inverse. *Do not use the MATLAB function* `inv( )`. Rather, take advantage of the structure of the matrix, e.g. that part of it is a rotation matrix.

Submission

- Submit your work as a single **zip** file on GradeScope. Name your submission using the format: **Lab1_YourJHED**.zip (e.g., Lab1_jkim115.zip).
- Inside your zip file, organize your files into the following two folders:
  - **Lab1_Code**: Include all your code files (scripts, functions, etc.) for this lab. We provide starter code in MATLAB. Make sure your helper and main files are complete and functional.
  - **Lab1_Written**: Include plots, figures, and written responses as annotated printouts (PDFs, scanned pages, or screenshots, as appropriate).
- Ensure that your code files are properly named, as specified in the lab instructions. For instance, when you are implementing a function called `ROTZ()`, it should be saved as `ROTZ.m`, not `rotz.m`.
- Before submitting, verify that your code runs as expected. If your main script depends on custom helper functions, be sure to include *all* relevant files in the Lab1_Code folder.
- To ensure your code runs smoothly during grading:
  - Avoid placing code files in subdirectories – keep everything flat.
  - Make sure that any relative paths in your code (e.g., for loading data or saving figures) are consistent with this folder structure.
- Submissions that do not follow these instructions will receive a point deduction, so please double-check your file organization and naming.